Programa de Ingeniería de Sistemas
Laboratorio de Ingeniería de Software II
Periodo 2-2026

Taller 10: Java Persistence API-JPA
(Será realizado en la Sala de Computo)

Objetivos

● El objetivo de este laboratorio es que los estudiantes comprendan cómo persistir un modelo de dominio utilizando JPA sin contaminar las entidades de negocio con anotaciones tecnológicas. A partir de un ejemplo existente, deberán analizar una solución por capas y aplicar los conceptos a un sistema de mayor complejidad: el Banco de Preguntas para las pruebas Saber Pro.
● El laboratorio tiene dos partes: primero, estudiar y comprender el demo de cursos, estudiantes y departamentos; después, diseñar e implementar una solución propia para el Banco de Preguntas del examen Saber Pro.

Reglas generales

● El código fuente, nombres de clases, métodos y comentarios técnicos deben estar en inglés.
● El código debe incluir documentación JavaDoc.
● Se debe utilizar Java 17 o superior, Spring Boot, Spring Data JPA, Maven y H2.
● Las clases del dominio no deben importar jakarta.persistence ni contener anotaciones JPA.
● Las interfaces de repositorio deben expresar contratos independientes de la tecnología.
● Las implementaciones concretas de esos contratos deben utilizar JPA.
● No se debe implementar arquitectura hexagonal. Se trabajará con una arquitectura en capas y separación de responsabilidades.

¿Qué se debe hacer?

Parte I. Comprensión del demo existente

Punto 1. Explorar y comprender el demo de cursos, estudiantes y departamentos

Material de partida: el proyecto **jpa-clean-persistence-demo.rar** entregado por el docente. Antes de programar nuevas funcionalidades, cada equipo debe descargar el proyecto, ejecutarlo y elaborar un análisis de su estructura y funcionamiento.

Actividad 1.1. Reconocimiento de la arquitectura
Identifiquen las clases que corresponden a cada capa y completen la siguiente tabla.

Elemento	Archivo o clase	Responsabilidad
Presentación	SchoolController	¿Qué hace?
Aplicación	SchoolService	¿Qué coordina?
Dominio	Student, Course, Department	¿Qué representan?
Contrato de repositorio	StudentRepository	¿Qué operaciones define?
Implementación concreta	StudentRepositoryJpa	¿Cómo persiste los datos?
Modelo JPA	StudentRecord	¿Qué representa?
Conversión de modelos	DomainMapper	¿Por qué es necesaria?

Tabla 1: Elementos del demo

Respondan las siguientes preguntas:

1. ¿Qué ocurriría si se agregara **@Entity** directamente a la clase de dominio **Student**?
2. ¿Por qué **SchoolService** depende de **StudentRepository** y no directamente de **SpringStudentDataRepository**?
3. ¿Qué responsabilidades tiene **StudentRepositoryJpa** que no deberían estar en **Student**?
4. ¿Qué ventajas y costos introduce tener una clase **Studen**t y otra **StudentRecord**?
5. ¿Qué tendría que cambiar si se reemplazara H2 por PostgreSQL manteniendo el mismo contrato de repositorio?

Actividad 1.2 Comprender las relaciones

Localicen las anotaciones correspondientes a las relaciones en las clases del paquete **infrastructure.jpa.record.**

Expliquen:

- ¿Cuál es el lado propietario de la relación entre **DepartmentRecord** y **CourseRecord**?
- ¿Por qué **mappedBy = "department"** aparece en la relación uno a muchos?

● ¿Qué función cumple **@JoinColumn** en la relación muchos a uno?
● ¿Qué tabla intermedia utiliza la relación muchos a muchos?
● ¿Qué diferencia existe entre la colección de cursos de un departamento y la colección de cursos de un estudiante?
Dibujen el esquema relacional resultante y señalen las claves primarias, las claves foráneas y la tabla intermedia.

Actividad 1.3. Comprender la herencia
Analicen **PersonRecord**, **StudentRecord** e **InstructorRecord**.
1. Expliquen qué hace **@Inheritance(strategy = InheritanceType.JOINED)**.
2. Identifiquen qué atributos se almacenan en la tabla base y cuáles en las tablas especializadas.
3. Comparen las clases de dominio con las clases JPA de la jerarquía.
4. Expliquen qué cambiaría conceptualmente si se utilizara **SINGLE_TABLE** o **TABLE_PER_CLASS**.

Actividad 1.4 Ejecutar y comprobar
Ejecuten las operaciones disponibles en el demo y comprueben los resultados en la consola H2.
Deben demostrar, como mínimo:
● Creación de un departamento.
● Creación de un curso asociado al departamento.
● Registro de un estudiante.
● Matrícula del estudiante en un curso.
● Consulta de los registros y de las tablas relacionadas.
Entregable de la parte I: un documento breve con la tabla de arquitectura completada, las respuestas justificadas y el diagrama del esquema relacional.

Patrón II: Diseño e implementación del Banco de Preguntas Saber Pro

Punto 2. Analizar el dominio del problema

El Banco de Preguntas Saber Pro es un sistema que permite construir, revisar, publicar y utilizar preguntas para preparar simulacros de evaluación. El sistema debe administrar usuarios, competencias, preguntas, opciones de respuesta, revisiones y cuestionarios.
Los actores principales son:
● Administrador: gestiona usuarios, roles y parámetros del sistema.
● Autor: crea y modifica preguntas.
● Revisor: evalúa la calidad y pertinencia de las preguntas.
● Docente: organiza cuestionarios y simulacros.
● Estudiante: responde preguntas y consulta sus resultados.
El sistema debe controlar el ciclo de vida de las preguntas:

DRAFT → PENDING_REVIEW → IN_REVIEW → APPROVED → PUBLISHED → ARCHIVED
Una pregunta también puede ser rechazada y regresar al estado de borrador para su corrección. Solo las preguntas aprobadas y publicadas pueden utilizarse en simulacros.

Punto 3. Diseñar el modelo de dominio

Antes de crear las clases Java, cada equipo debe elaborar un diagrama de clases UML que represente las entidades, atributos, asociaciones y restricciones del sistema.
La siguiente propuesta sirve como guía de diseño, no como una solución que deba copiarse literalmente.

Entidades sugeridas
Usuarios y permisos
User, Role, UserProfile
Representan a las personas que interactúan con el sistema, sus roles y sus datos complementarios.

Clasificación académica
Competency, Topic, QuestionTag
Organizan las preguntas por competencias, temas y etiquetas.

Banco de preguntas
Question, MultipleChoiceQuestion, TrueFalseQuestion, AnswerOption, QuestionMetadata
Representan el contenido, el tipo de pregunta, sus opciones y sus metadatos.

Revisión y publicación
Review, ReviewCriterion, ReviewResult, QuestionHistory
Permiten registrar evaluaciones, resultados y cambios realizados durante el ciclo de vida.

Evaluación
Quiz, QuizQuestion, Attempt, StudentAnswer, Result
Permiten crear cuestionarios, presentar preguntas y registrar los resultados de los estudiantes.
No es necesario implementar todas estas clases en el mismo nivel de detalle. Cada equipo debe justificar cuáles necesita para su alcance y cómo se relacionan.

Punto 4. Aplicar los cuatro tipos de relaciones

El diseño debe contener, como mínimo, una relación de cada tipo. Se recomienda utilizar los siguientes casos de negocio.

4.1. Relación uno a uno — @OneToOne

Propuesta: Question y QuestionMetadata.

Cada pregunta tiene un único registro de metadatos complementarios y cada registro pertenece a una única pregunta.

Ejemplos de metadatos: fecha de última revisión, nivel cognitivo, tiempo estimado de respuesta y observaciones editoriales.

El equipo debe decidir:

- Qué entidad es propietaria de la relación.
- Dónde se almacenará la clave foránea.
- Si la relación es obligatoria u opcional.
- Cómo garantizar que una pregunta no tenga varios registros de metadatos.

4.2. Relación uno a muchos — @OneToMany

Propuesta: **Question** y **AnswerOption**.

Una pregunta de selección múltiple o única puede tener varias opciones de respuesta. Cada opción pertenece a una sola pregunta.

En el diseño inicial de las preguntas de selección única, contemplen cuatro distractores y una respuesta correcta, de acuerdo con las reglas académicas del sistema.

El equipo debe definir:

- Cómo representar las opciones.
- Cómo impedir que una opción quede asociada a una pregunta inexistente.
- Cómo controlar el número de opciones y la respuesta correcta.
- Qué ocurre con las opciones cuando una pregunta se elimina.

4.3. Relación muchos a uno — @ManyToOne

Propuesta: **Question** y **User**.

Cada pregunta tiene un autor, mientras que un autor puede crear muchas preguntas.

También pueden modelar la relación entre Question y Topic: muchas preguntas pueden pertenecer a un mismo tema.

Deben identificar el lado propietario de la asociación, la clave foránea y las reglas que impiden asignar un autor inexistente.

4.4. Relación muchos a muchos — @ManyToMany

Propuesta: **Question** y **Competency**.

Una pregunta puede evaluar varias competencias, y una competencia puede estar relacionada con muchas preguntas.

Ejemplo: una pregunta de arquitectura de software podría contribuir a evaluar tanto el diseño de

soluciones como el análisis de problemas.
El equipo debe definir:
- El nombre de la tabla intermedia.
- Sus dos claves foráneas.
- Si la asociación necesita atributos adicionales, como el peso de una competencia en la pregunta.
- Si conviene mantener **@ManyToMany** o convertir la tabla intermedia en una entidad propia.

Reto de diseño: si la relación necesita guardar el porcentaje de contribución de una competencia, propongan una entidad asociativa como **QuestionCompetency**. En ese caso, la asociación se modelaría con dos relaciones uno a muchos y dos muchos a uno.

Punto 5. Aplicar herencia en el modelo de preguntas

El sistema debe soportar diferentes tipos de preguntas. Para este ejercicio, utilicen una clase base **Question** y, como mínimo, dos especializaciones.

Question
id, statement, difficulty, status, author

MultipleChoiceQuestion
Opciones de respuesta y respuesta correcta.

TrueFalseQuestion
Valor de verdad esperado.

La jerarquía anterior representa el concepto de herencia. No significa que todas las clases del dominio deban ser entidades JPA.
Cada equipo debe crear una jerarquía de dominio sin anotaciones de persistencia y una jerarquía de persistencia independiente que implemente el mapeo.
Comparen las estrategias:
- **SINGLE_TABLE**: toda la jerarquía se almacena en una tabla con una columna discriminadora.
- **JOINED**: los atributos comunes se almacenan en una tabla base y los específicos en tablas de subclase.
- **TABLE_PER_CLASS**: cada clase concreta dispone de una tabla con los atributos heredados y propios.

Seleccionen una estrategia para el proyecto y justifiquen su elección según la estructura de las preguntas y las consultas que esperan realizar.

Parte III. Arquitectura en capas y persistencia

Punto 6. Definir la arquitectura de la solución

El proyecto debe organizarse en paquetes que separen las responsabilidades. Se propone la siguiente estructura inicial:

```
src/main/java/com/example/questionbank/
├── api/
│   ├── QuestionController.java
│   └── QuizController.java
├── application/
│   ├── QuestionService.java
│   ├── ReviewService.java
│   └── QuizService.java
├── domain/
│   ├── model/
│   │   ├── Question.java
│   │   ├── MultipleChoiceQuestion.java
│   │   ├── TrueFalseQuestion.java
│   │   ├── User.java
│   │   └── Competency.java
│   └── repository/
│       ├── QuestionRepository.java
│       ├── UserRepository.java
│       └── CompetencyRepository.java
└── infrastructure/
    └── jpa/
        ├── record/
        ├── repository/
        ├── mapper/
        └── *RepositoryJpa.java
```

Los paquetes son una propuesta inicial: pueden ampliarse según el diseño final del equipo.

Reglas de dependencia

1. La capa **api** recibe las solicitudes y delega los casos de uso.
2. La capa **application** coordina operaciones y utiliza contratos de repositorio.
3. El paquete **domain** contiene el modelo de negocio y las interfaces abstractas de persistencia.
4. La infraestructura implementa los contratos mediante Spring Data JPA.
5. Los mapeadores traducen entre los objetos de dominio y las entidades JPA.

No se permite que una entidad de dominio tenga anotaciones como **@Entity, @Table, @Id, @OneToMany** o **@ManyToMany**.

Tampoco se permite que los servicios de aplicación dependan directamente de **JpaRepository**.

Punto 7. Implementar los repositorios

Cada equipo debe definir contratos de repositorio para las entidades principales y crear implementaciones concretas con JPA.
Por ejemplo, el contrato de **QuestionRepository** debería permitir:

- Guardar una pregunta.
- Consultar una pregunta por identificador.
- Listar preguntas.
- Consultar preguntas por autor.
- Consultar preguntas por estado.
- Consultar preguntas publicadas por competencia.

No es obligatorio implementar todas las consultas si el alcance del equipo es reducido, pero sí deben implementar las operaciones necesarias para demostrar los casos de uso elegidos.
Pregunta de reflexión: ¿por qué es preferible que **QuestionService** invoque **QuestionRepository** y no directamente **SpringQuestionDataRepository**?

Punto 8. Implementar casos de uso

El equipo debe implementar, como mínimo, los siguientes casos de uso:
1. Crear una pregunta. Registrar su enunciado, autor, clasificación y tipo, junto con sus opciones si corresponde.
2. Consultar preguntas. Recuperar preguntas por identificador y listar preguntas de acuerdo con algún criterio.
3. Clasificar una pregunta. Asociarla a una o varias competencias y a un tema.
4. Enviar una pregunta a revisión. Validar que cumpla las condiciones mínimas para pasar del estado de borrador a pendiente de revisión.
5. Crear un cuestionario. Asociar varias preguntas publicadas y garantizar que no se incorporen preguntas que todavía no estén aprobadas y publicadas.

Para el cambio de estado, no basta con actualizar una columna arbitrariamente: el servicio debe comprobar las reglas de negocio y utilizar transacciones cuando la operación lo requiera.

Punto 9. Probar la persistencia con H2

La aplicación debe ejecutarse con una base de datos H2 en memoria (por facilidad).
Los equipos deben demostrar mediante pruebas automatizadas o pruebas de integración que:

- Las entidades se guardan y se recuperan correctamente.
- Las relaciones se conservan al consultar los objetos.
- La herencia permite persistir y recuperar correctamente los diferentes tipos de pregunta.
- Las claves foráneas y restricciones impiden relaciones inválidas.
- Las reglas del ciclo de vida evitan utilizar preguntas no publicadas en cuestionarios.
- Los contratos de repositorio pueden probarse sin acoplar las pruebas de dominio a JPA.

Incluyan casos de éxito y casos de error. No es suficiente demostrar que la aplicación inicia correctamente.

Parte IV. Entregables y evaluación

Entregables por equipo.

Lista de verificación

- Informe de análisis del demo inicial.
- Diagrama de clases UML del Banco de Preguntas.
- Diagrama del modelo relacional con claves primarias y foráneas.
- Código fuente completo en Java con JavaDoc.
- Implementación de los contratos e implementaciones JPA.
- Configuración de Spring Boot y H2.
- Pruebas automatizadas o de integración.
- README.md con propósito, arquitectura, instrucciones de ejecución y ejemplos de uso.

Rúbrica de evaluación

Criterio	**Porcentaje**
Comprensión del demo y análisis de la arquitectura	15 %
Diseño del dominio y diagrama UML	20 %
Mapeo de relaciones y herencia con JPA	25 %
Separación de responsabilidades y repositorios abstractos	20 %

Pruebas, ejecución y validación de reglas de negocio	10 %
Documentación y calidad del código	10 %
Total	100 %

Preguntas de reflexión para la sustentación

1. ¿Qué ventajas y desventajas tiene mantener dos modelos, uno de dominio y otro de persistencia?
2. ¿Qué cambios serían necesarios si el sistema dejara de utilizar JPA?
3. ¿Cuándo conviene una relación muchos a muchos directa y cuándo es mejor crear una entidad asociativa?
4. ¿Qué problemas pueden aparecer al recuperar asociaciones perezosas (LAZY) fuera de una transacción?
5. ¿Qué estrategia de herencia elegirían si el sistema incorpora muchos tipos de preguntas con atributos y reglas diferentes?
6. ¿Por qué la persistencia de una pregunta y la validación de su ciclo de vida son responsabilidades distintas?

Criterio final: no se evaluará únicamente que la aplicación funcione. Cada equipo debe demostrar que comprende por qué organizó así el modelo de dominio, cómo JPA representa las relaciones y de qué manera la arquitectura evita que las decisiones de persistencia contaminen las reglas de negocio.

Fecha de entrega: martes, 27 de octubre de 2026 hasta las 18:00

Referencias

Material de apoyo:
Wilson L Pantoja, Julio A Hurtado.